

# PRAGMA: Revolut Foundation Model

Maxim Ostroukhov<sup>1</sup> Ruslan Mikhailov<sup>1</sup> Vladimir Iashin<sup>1</sup>  
 Artem Sokolov<sup>1</sup> Andrei Akshonov<sup>1</sup> Vitaly Protasov<sup>1</sup> Dmitrii Beloborodov<sup>1</sup>  
 Vince Mullin<sup>2</sup> Roman Y. Enzmann<sup>2</sup> Georgios Kolovos<sup>2</sup> Jason Renders<sup>2</sup>  
 Pavel Nesterov<sup>1</sup> Anton Repushko<sup>1</sup>

<sup>1</sup>Revolut Research <sup>2</sup>NVIDIA

Modern financial systems generate vast quantities of transactional and event-level data that encode rich economic signals. This paper presents PRAGMA, a family of foundation models for multi-source banking event sequences. Our approach pre-trains a Transformer-based architecture with masked modelling on a large-scale, heterogeneous banking event corpus using a self-supervised objective tailored to the discrete, variable-length nature of financial records. The resulting model supports a wide range of downstream tasks such as credit scoring, fraud detection, and lifetime value prediction: strong performance can be achieved by training a simple linear model on top of the extracted embeddings and can be further improved with lightweight fine-tuning. Through extensive evaluation on downstream tasks, we demonstrate that PRAGMA achieves superior performance across multiple domains directly from raw event sequences, providing a general-purpose representation layer for financial applications.

**Disclaimer:** We report only relative improvements, as absolute metrics are commercially sensitive. All examples are synthetic and not from real production data.

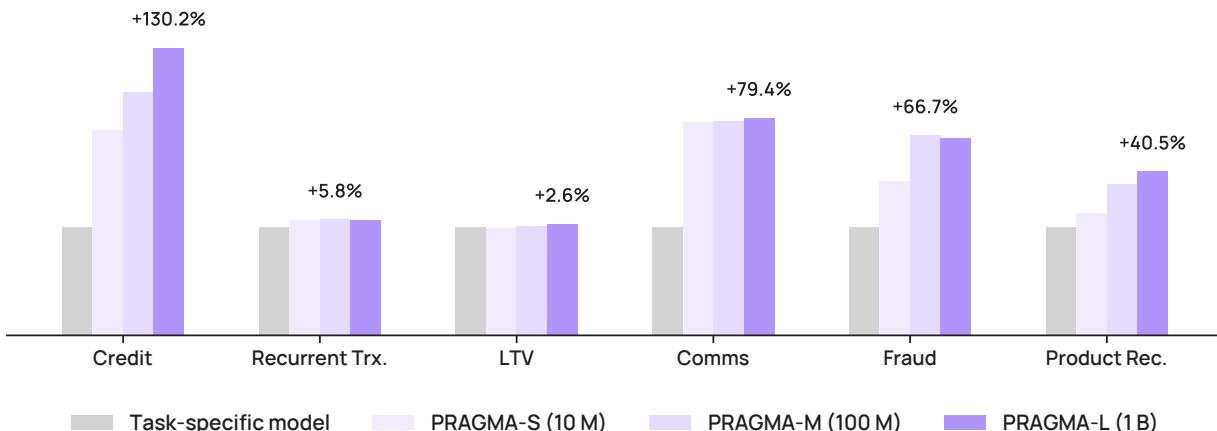


Figure 1: A single architecture from 10M to 1B parameters that outperforms task-specific models across tasks.

## 1 Introduction

Foundation models are general-purpose models trained at scale on broad data distributions and subsequently adapted to a wide variety of downstream tasks (Bommasani et al., 2021). While such models have transformed natural language processing (Devlin et al., 2019; Brown et al., 2020) and computer vision (Kirillov et al., 2023; Caron et al., 2021), their application to multi-source banking user histories remains comparatively underexplored. Modern banks and fintechs accumulate large volumes of data: event streams spanning card and transfer transactions, product usage, in-app navigation, and customer communications, alongside static generalised profile state such as account tenure and

plan. These event streams encode signals relevant to risk management, product analytics, and operations, but they are difficult to model efficiently with off-the-shelf language-model tokenisation and architectures. While serialising structured records as text and feeding them to a standard Transformer is a viable baseline, it inflates sequence lengths considerably because every field name and delimiter becomes several subword tokens. Moreover, numerical values are split into digit fragments that discard magnitude and ordering, both of which are critical for financial reasoning. Together, these limitations make naive text serialisation impractical for the long, heterogeneous user histories common in banking.

Multi-source banking user histories differ from text in three ways. First, each event is a variable-length record with mixed categorical, numerical, and free-text fields. Second, histories are long-tailed in length and irregular in time, with strong daily and weekly cycles. Third, practical deployments must operate under strict privacy and regulatory constraints, which limit what can be reported and which features can be used for certain decisions. Because no single off-the-shelf architecture handles all three challenges simultaneously, practitioners default to building task-specific pipelines with extensive feature engineering, making it hard to share statistical strength across domains and products.

Prior work addresses isolated slices of this problem. Tabular Transformers such as TabTransformer and FT-Transformer (Huang et al., 2020; Gorishniy et al., 2021) model fixed-schema rows, while sequential recommender models such as SASRec and BERT4Rec (Kang et al., 2018; Sun et al., 2019) operate on item-like interaction histories. Financial foundation models have largely focused on text or generic time-series tokenisation (Yang et al., 2020; Wu et al., 2023; Yang et al., 2023; Jin et al., 2024; Ansari et al., 2024), while newer transaction-ledger models such as nuFormer and TransactionGPT (Braithwaite et al., 2025; Dou et al., 2025) move closer to our setting. However, these models typically ingest a single event source, omit static profile state, and are evaluated on a narrow set of tasks: nuFormer targets product recommendation, while TransactionGPT focuses on anomaly detection and trajectory generation. The literature still lacks a multi-source encoder backbone with explicit profile state that transfers across a broad range of discriminative banking tasks.

In this paper, we present PRAGMA, a family of encoder-style foundation models for multi-source banking user histories. PRAGMA is pre-trained with masked modelling on a large-scale corpus of user histories that combines multi-source events with static profile state (§2.1). To handle heterogeneity, we apply a key–value–time tokenisation scheme with type-specific value encoding for numerical, categorical, and textual fields (§2.2). The resulting backbone uses two encoder branches for profile state and events whose outputs are fused by a history encoder (§2.3).

We choose an encoder-only, bidirectional design because our primary goal is transferable representations for discriminative financial tasks, rather than open-ended generation. Masked modelling enables each token to attend to both past and future context (Devlin et al., 2019), which is particularly useful when reconstructing partially observed event records and learning record-level representations from complete histories. After pre-training, PRAGMA can be adapted efficiently in two complementary ways (§3.1). In the *embedding probe* setting, we freeze the backbone and train a lightweight head on top of the extracted embeddings. In the *LoRA fine-tuning* setting, we apply Low-Rank Adaptation (LoRA) (Hu et al., 2022) to update only a small fraction of parameters, enabling fast specialisation while keeping most of the backbone shared across tasks.

We evaluate PRAGMA on a suite of internal downstream benchmarks spanning credit scoring, fraud detection, communication engagement, recurrent transaction detection, lifetime value prediction, and more (§3.2). Across evaluated domains, PRAGMA consistently outperforms strong task-specific baselines while reducing the need for hand-crafted features (Figure 1). We further describe the engineering choices required to train PRAGMA efficiently on long and highly variable user histories, including sequence packing and dynamic batching (§2.4).

Our contributions are as follows:

- We introduce PRAGMA, a family of encoder-style foundation models for multi-source banking user histories, scaling from 10 M to 1 B parameters, to our knowledge, the largest published encoder backbone for consumer banking event sequences. The architecture combines a key–value–time tokenisation scheme with a two-branch design in which profile-state and event encoders feed a history encoder for heterogeneous financial records.
- We describe an efficient pre-training recipe for long and irregular banking user histories based on masked modelling, sequence packing, and dynamic batching, and show that LoRA fine-tuning of a pre-trained backbone consistently matches or outperforms full training from scratch.

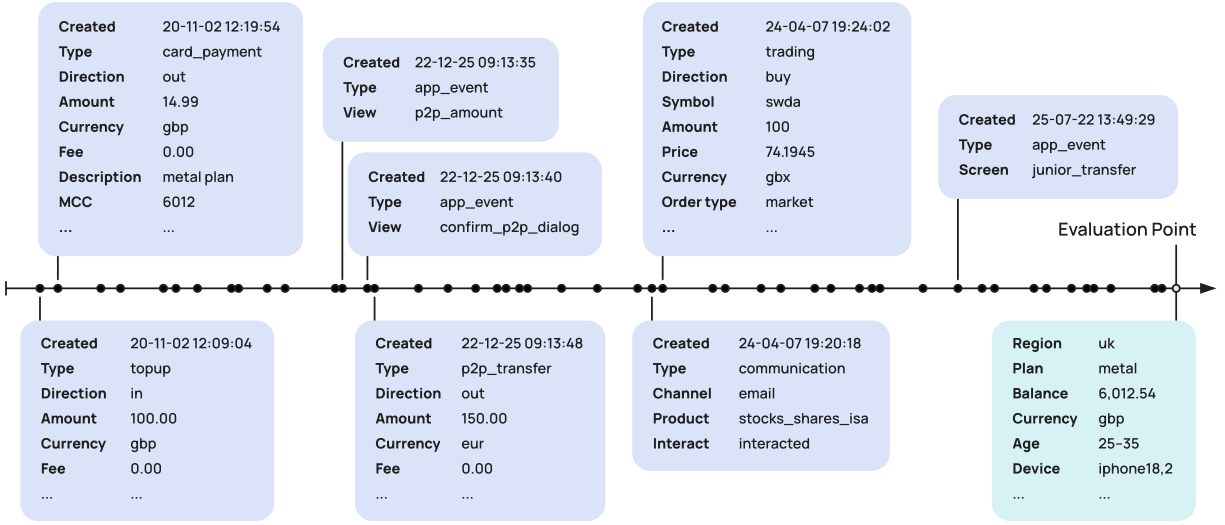


Figure 2: **Event timeline overview.** After account creation, users generate a sequence of platform interactions over time, spanning transactions, in-app navigation, and communications. We aggregate the event history up until a designated evaluation point. Alongside these sequential events, we capture contextual attributes that describe the record’s state at that point, e.g., membership plan or service region. Both events and attributes share a uniform representation: a timestamp and a set of key–value pairs (e.g., `Type: card_payment`, `Channel: email`). All values shown are synthetic; the figure is for illustration purposes only.

- We evaluate a single pre-trained backbone across six diverse downstream tasks (credit scoring, fraud detection, lifetime value, communication engagement, recurrent transaction detection, and product recommendation), a substantially broader task scope than prior transaction-ledger models, which typically target one or two tasks. PRAGMA consistently outperforms strong task-specific baselines while reducing the need for hand-crafted features.

## 2 Pre-training

### 2.1 Dataset

Our goal is to build a foundation model that encodes diverse event-level signals and transfers across a wide range of downstream tasks. Our dataset is structured at the record level, where each observation represents a pseudonymised event history associated with an evaluation point. As shown in Figure 2, we consider an event history alongside contextual attributes. This approach enables the model to account for both sequential patterns and time-invariant features like account currency.

All data used in this work is fully anonymised and contains no personally identifiable information. We construct our pre-training dataset from 26 M user records spanning 111 countries, accumulating 24 B events that total 207 B tokens.

#### 2.1.1 Event History

Standard platform usage generates event streams across various services, e.g., account funding, payments, in-app navigation, or service communications. These aggregated event histories capture population-level patterns that support a range of analytical and predictive tasks. An event is defined by a created timestamp and a set of key–value pairs, e.g., `Direction: out`. We fetch events from broad source types that can be loosely grouped into transactions, app, trading, and communication, which were selected for their high expected impact on downstream tasks. Event schemas are specific to their source type and incorporate distinct sets of keys, e.g., `Symbol` key is unique to trading events. Beyond anonymisation, de-identification, and standard eligibility criteria, no additional statistical filtering or

pre-processing, such as outlier removal or vocabulary pruning, is applied to the event streams, to ensure that the model captures the **full heterogeneity found in production**.

### 2.1.2 Profile State

In addition to the event history, we incorporate general contextual attributes such as **balance quantile, plan, insurance state, and service region**. These attributes provide useful context that is otherwise missing from the event history alone. Profile state is a set of descriptive key-value pairs in an event-like format, e.g., `Plan: metal`, timestamped at the designated evaluation point (or the cut-off date during pre-training).

High-activity users often generate tens of thousands of interactions, exceeding computational bounds; we address this via truncation to a fixed context window (§2.3.5). However, truncation risks discarding early historical milestones that carry useful signal, such as account age. We therefore augment profile state with **life-long events**, key-value pairs that, unlike regular profile attributes, each carry an individual timestamp recording a first occurrence, e.g., **Lifelong: first\_topup at 20-11-02 12:09:04**. This timestamp is then used to compute the temporal distance to the evaluation point, enabling the model to encode the timing of historical milestones.

### 2.1.3 Pre-training Time Range

Developing a robust and generalisable model requires a delicate balance between maximising historical coverage and maintaining data relevance. Accordingly, determining the optimal temporal range for pre-training involves navigating several trade-offs between event diversity, distribution shift, and computational efficiency.

First, simply including every event from the full available dataset is often impractical and sub-optimal. Older events may reflect historical patterns, product features, or system dynamics that are no longer relevant at inference time. Such discrepancies create a distribution mismatch that can degrade performance, as the model may struggle to generalise from obsolete historical examples to the evolving behaviours present in deployment. Additionally, the inclusion of highly heterogeneous events from long time spans can make the pre-training task harder and slow down model convergence. Second, downstream applications may require making predictions on events that took place within temporal ranges either much earlier or much later than those used for pre-training. If the model is not exposed to sufficient diversity in both recent and less-common historical patterns, the performance on these out-of-distribution inputs may suffer. Finally, Transformer architectures have a limited effective context span, determined both by model design and hardware constraints.

With these considerations in mind, we select a temporal range of 25 months from 2023 to 2025 for pre-training, balancing comprehensive event coverage, recency, distribution consistency, and tractable sequence modelling.

## 2.2 Tokenisation

Unlike standard LLMs that treat everything as text, a financial foundation model needs to preserve the structural nature and heterogeneity of tabular data. We address this challenge by implementing a disentangled embedding space of input tokens.

As shown in Figure 3, we represent each data point by three components: a semantic type (key), a value, and a temporal coordinate, following a common standard in tabular event data (Braithwaite et al., 2025). For instance, `Channel: email at 24-04-07 19:20:18` maps to a key, a value, and a temporal coordinate, respectively. This ensures that the model distinguishes between the meaning of a field and its value, while also encoding event chronology. Next, we present how the three are tokenised.

**Semantic Type (Key).** The semantic type embedding enables the model to learn the meaning of a field and to contextualise the value it holds. We tokenise all semantic types (keys) as single tokens, and both event and profile state semantic types are encoded in a similar way. This results in a vocabulary of  $\sim 60$  tokens.

**Value.** We cover the diversity of values with three value types: *numerical*, *categorical*, and *textual*. Numerical values are mapped to **percentile buckets, where bin boundaries are learned from training data with an extra bucket for zero**, allocating one token per bucket. The distinction between categorical and textual is determined by cardinality thresholding: string fields whose number of unique values falls below a predefined threshold are treated as categorical,

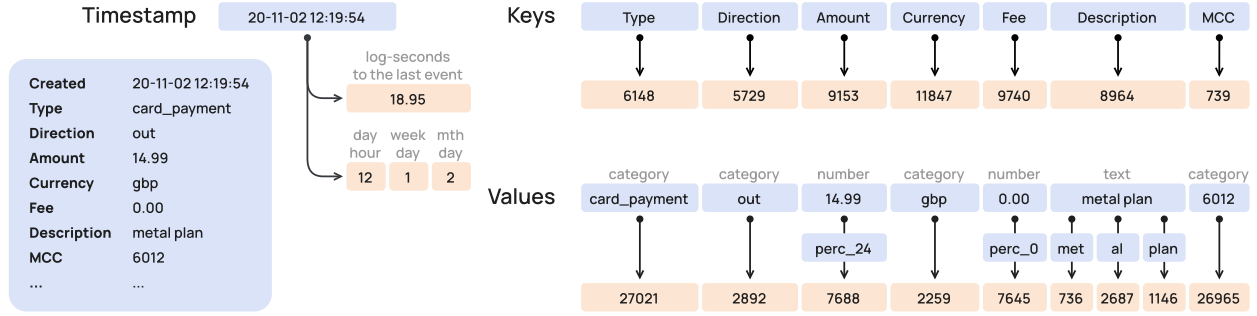


Figure 3: **Tokenisation overview.** A raw event record is decomposed into a temporal coordinate, semantic types (keys), and values. Keys are always represented by one token, while values use type-specific tokenisation: numerical values are bucketised by percentile, categorical values map to a single token, and textual values are split into subword tokens. Some keys therefore expand to multiple value tokens, e.g., Description  $\rightarrow$  met, al, plan. Time is encoded both as log-seconds to the last event and as calendar and time features derived from the timestamp. Profile state is encoded similarly to an event record.

while higher-cardinality fields are treated as textual. Categorical values are manually selected from all text fields to prevent splitting common values, such as merchant category codes (MCC), into multiple tokens, and are represented as a single token as well. For textual fields, values are tokenised with a BPE-style subword tokeniser (Sennrich et al., 2016) with a reserved [UNK] token for rare unseen fragments. In total, values allocate a vocabulary of  $\sim 28$  k tokens.

**Temporal Information.** We encode time in two ways. First, we compute the elapsed time since the most recent event, measured in seconds. We then apply a soft logarithmic transformation,  $8 \cdot \ln(1 + t/8)$ , to compress the dynamic range of *life-long* events while preserving high-resolution linear granularity for recent events. This prevents aliasing in positional embeddings caused by extreme temporal gaps without sacrificing the precision of local event sequencing. Second, to capture daily and weekly temporal cycles, we additionally decompose each event timestamp into its cyclical constituents: hour of day, day of week, and day of month, and embed them using periodic functions similar to Gorishniy et al. (2022), but with periods fixed to the known calendar cycles rather than learned. Calendar features are applied only to event-history entries, as cyclical patterns are less relevant for one-off life-long events where the log-seconds encoding already captures the relevant temporal signal.

### 2.3 Model Architecture

PRAGMA is an encoder-only Transformer that inputs an event history along with contextual attributes and outputs dense record-level embeddings. It is trained on a large-scale, diverse dataset with a masked modelling (MLM) objective that reconstructs masked input tokens. Once pre-trained, it acts as a backbone for downstream adaptation with small-scale (2–4 % of the model’s parameters) fine-tuning for a variety of tasks. An overview of PRAGMA is shown in Figure 4.

PRAGMA is parametrised as a family of models with 10 M, 100 M, and 1 B parameters, enabling selection according to operational budget and constraints. The details of the architecture family are provided in Table 1. All size variants use GELU activations (Hendrycks et al., 2016), pre-norm layer normalisation (Xiong et al., 2020), and dropout of 0.1 (Srivastava et al., 2014).

The model consists of three main blocks: Profile State Encoder, Event Encoder, and History Encoder. First, the profile state tokens are processed by the Profile State Encoder. Second, similar to profile state, each event is encoded independently in the Event Encoder. Finally, the outputs of the Profile State and Event Encoders are concatenated and encoded in the History Encoder to form an output. Depending on the stage, the final output is used either in an MLM head during pre-training, a classification head during fine-tuning, or as-is in an embedding probe.

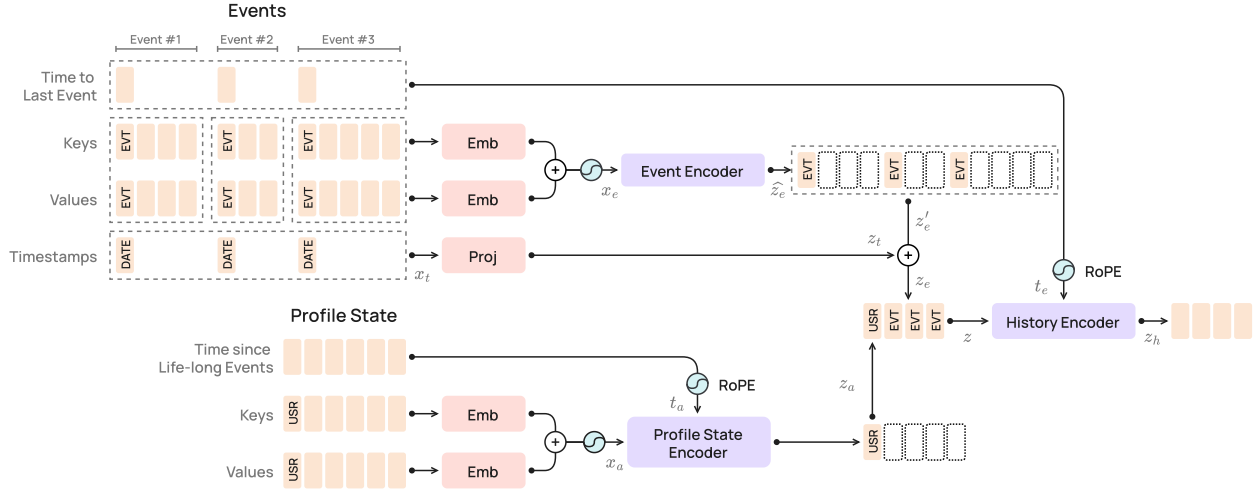


Figure 4: **PRAGMA backbone overview.** Each user record is represented as an ordered event history and profile state, where every field is decomposed into a semantic type (key), one or more values, and a temporal coordinate. Keys and values are embedded from a shared lookup table, and value tokens receive within-field positional embeddings. A *Profile State Encoder* maps profile state  $x_a$ , with time since life-long events  $t_a$  encoded via RoPE, into a [USR] embedding  $z_a$ , while an *Event Encoder* independently maps the tokens of each event  $x_e$  into a [EVT] embedding  $z'_e$  and adds calendar features  $z_t$ . A *History Encoder* then contextualises the sequence  $z = [z_a : z_e]$  with time to the last event  $t_e$  encoded via RoPE, producing a representation for a user record  $z_h$ .

| Model    | Params | Width              |                  | Depth   |       |         |       |
|----------|--------|--------------------|------------------|---------|-------|---------|-------|
|          |        | $d_{\text{model}}$ | $d_{\text{ffn}}$ | Profile | Event | History | Heads |
| PRAGMA-S | 10 M   | 192                | 768              | 1       | 5     | 2       | 3     |
| PRAGMA-M | 100 M  | 512                | 2048             | 3       | 16    | 6       | 8     |
| PRAGMA-L | 1 B    | 1024               | 4096             | 9       | 45    | 18      | 16    |

Table 1: **PRAGMA model family.** PRAGMA scales across three variants (10 M, 100 M, 1 B parameters) by jointly increasing model width ( $d_{\text{model}}$ ,  $d_{\text{ffn}}$ ), depth of the profile-state, event, and history encoders, and the number of attention heads.

### 2.3.1 Token Embedding

Profile state and event tokens are embedded identically. For multi-valued fields (e.g., *Description*), the key token is replicated to match each of its values, yielding  $n$  key–value pairs in total. A single shared embedding table  $E$  maps each key and value to a  $d$ -dimensional vector; the two embeddings are summed and augmented with static sine/cosine positional encodings (PosEmb) (Vaswani et al., 2017):

$$x = \text{PosEmb}(E(k) + E(v)), \quad x \in \mathbb{R}^{n \times d}. \quad (1)$$

Positions index values *within* a field, not across fields—e.g., the value `eur` of *Currency* receives position 0, while the three value tokens (`met`, `al`, `plan`) of *Description* receive positions (0, 1, 2) (see Figure 3). We denote user and event embeddings as  $x_a \in \mathbb{R}^{n_a \times d}$  and  $x_e \in \mathbb{R}^{n_e \times d}$ , respectively. Following common practice in encoder-only Transformers (Devlin et al., 2019; Dosovitskiy et al., 2021), a learnable [USR] (or [EVT]) token is prepended to each sequence (Figure 4).

### 2.3.2 Profile State Encoder

The Profile State Encoder is a bidirectional Transformer. It inputs the profile state tokens  $x_a \in \mathbb{R}^{n_a \times d}$  and corresponding temporal coordinates  $t_a \in \mathbb{R}^{n_a}$ , where each entry holds the log-seconds since the corresponding life-long event (or 0 for non-life-long pairs). We use RoPE (Su et al., 2024) to encode  $t_a$ . We disentangle this positional embedding



from the value-level positional embedding discussed in §2.3.1 to avoid the semantic and scale mismatch. The output is a sequence of profile state embeddings  $z_a \in \mathbb{R}^{n_a \times d}$ . We pass the first element, which corresponds to the [USR] token, to the History Encoder—we refer to it as  $z_a \in \mathbb{R}^{1 \times d}$  for simplicity.

### 2.3.3 Event Encoder

The Event Encoder is a bidirectional Transformer, similar to the Profile State Encoder. It inputs an event history  $x_e = (x_{e,1}, x_{e,2}, \dots, x_{e,n_e})$ , where each element has a distinct number of token embeddings ( $x_{e,i} \in \mathbb{R}^{n_i \times d}$ ), and processes each event independently of all other events in the history. The module outputs a token-level embedding sequence for each event, denoted  $\hat{z}_e$ , which is used by the MLM head during pre-training. Similar to the Profile State Encoder, we select the first token corresponding to the [EVT] token for each event as its aggregated representation  $z'_e \in \mathbb{R}^{n_e \times d}$ .

The calendar features (hour of day, day of week, and day of month)  $x_t \in \mathbb{R}^{n_e \times 3}$  are converted to sine and cosine radians and embedded with two MLP layers into  $z_t \in \mathbb{R}^{n_e \times d}$ . Next, the embedded calendar features are added to the Event Encoder output:  $z_e = z'_e + z_t$ .

### 2.3.4 History Encoder

The History Encoder is a bidirectional Transformer, similar to the other two encoders. It inputs the concatenated aggregated representations of profile state and the calendar-augmented events:  $z = [z_a : z_e] \in \mathbb{R}^{(1+n_e) \times d}$ , as well as the corresponding temporal coordinate  $t_e \in \mathbb{R}^{1+n_e}$ , where each entry holds the log-seconds to the most recent event in the history (0 for the  $z_a$  position). Similar to the Profile State Encoder, RoPE is used to encode positional information. The output is a sequence of embeddings  $z_h \in \mathbb{R}^{(1+n_e) \times d}$ , where  $z_{h,0}$  corresponds to [USR] and  $z_{h,1}, \dots, z_{h,n_e}$  to the [EVT] tokens.  $z_h$  is used by the MLM head during pre-training and for downstream probes.

### 2.3.5 Training

**Pre-training Objective.** PRAGMA is pre-trained with an MLM objective following BERT (Devlin et al., 2019) where a random subset of event input tokens is masked, and the model reconstructs the original tokens. For each masked token, the MLM head receives the concatenation of three  $d$ -dimensional vectors: the Event Encoder output at that token’s position within  $\hat{z}_e$ , providing local within-event context; the History Encoder output at the corresponding [EVT] position  $z_{h,i}$ , providing cross-event context; and the History Encoder output at the [USR] position  $z_{h,0}$ , providing user-level context. This  $3d$ -dimensional representation is projected back to  $d$  dimensions and matched against the embedding table to produce logits. The training loss is cross-entropy with label smoothing (Szegedy et al., 2016).

**Masking Strategy.** The masking strategy combines three sources: standard individual token-level masking (with 15 % probability), event-level masking (10 %) that requires the model to reconstruct an entire event, and semantic-type (key)-level masking (10 %) where all values of the selected keys are masked, training the model to predict values given context and a key. During pre-training, a small fraction of selected positions are replaced with [UNK] rather than [MASK]. Because [UNK] positions are excluded from the MLM objective, they receive no gradient and effectively act as a form of input dropout, training the model to recover original values under a stronger corruption scheme and reducing reliance on the presence of [MASK], which does not occur at inference time.

**Downstream Adaptation.** PRAGMA supports two modes of downstream adaptation. In the *embedding probe mode*, the record-level representation produced by the History Encoder is extracted as a frozen feature vector, and a lightweight linear probe is trained on top. In the *LoRA fine-tuning mode*, a small fraction ( $\sim 2\text{--}4\%$ ) of model weights (the attention and feed-forward projections) are updated via Low-Rank Adaptation (Hu et al., 2022), keeping the pre-trained backbone mostly frozen and reducing the risk of catastrophic forgetting.

## 2.4 Training Infrastructure

Pre-training PRAGMA on 207 B tokens spanning 24 B user events introduces several engineering challenges. The heterogeneous, table-structured nature of the data requires specialised storage, batching, and truncation strategies. We describe each in turn below.

**Data Storage.** The pre-training corpus is stored as a two-level structure: a *user index* (an LMDB-backed key-value store mapping each user to their tokenised profile state and per-user token statistics) and a collection of *event shards* (Parquet files partitioned by event count, so each file contains only users with the same number of events). This layout allows workers to stream event shards independently and look up profile state on demand.

**Batching.** Each training sample consists of a complete event history together with its associated profile state tokens. Because event histories vary greatly in length, from a handful of events to thousands, naïve padding-based batching would waste the majority of compute on padding tokens. Sharding records by event count avoids many random-access disk operations during loading and yields uniform-length event sequences within each batch, so the History Encoder operates on a rectangular tensor without ragged or padded dimensions. We employ *dynamic batching* with a fixed token budget that fits into GPU memory: records from the same shard are greedily packed until the budget is reached.

**Sequence Packing.** Within a batch, individual events still vary in their number of tokens. Rather than padding every event to the longest one, we pack all event tokens into a flat buffer and process them with a variable-length (varlen) attention kernel (Dao et al., 2022), so tokens from different events do not attend to each other at this stage. Together with shard-based batching, this eliminates padding overhead along both the event and token axes. Compared to a padded baseline, sequence packing coupled with dynamic batching yields a 2–5 $\times$  throughput improvement, depending on the sequence length distribution in the dataset.

**Truncation.** To bound memory consumption at a fixed context length, we apply two levels of truncation before packing. At the *event level*, each individual event is truncated to at most 24 tokens, affecting only 0.01 % of events. At the *profile state level*, the static profile state sequence is truncated to at most 200 tokens. Users with zero events are discarded; users with more than 6,500 events are subsampled by retaining the most recent ones, preserving temporal recency.

**Pre-training Compute.** The three model variants were trained with bf16 mixed precision and the Muon optimiser combined with AdamW (Loshchilov et al., 2019; Jordan, 2024; Liu et al., 2025). PRAGMA-S (10 M parameters) and PRAGMA-M (100 M) were trained on 16 $\times$  NVIDIA H100 GPUs, and PRAGMA-L (1 B) on 32 $\times$  NVIDIA H100 GPUs. The smallest variant converged in approximately 2 days, while the 100 M and 1 B models each required roughly 2 weeks of wall-clock time.

## 3 Evaluation

For commercial sensitivity reasons, we do not report absolute downstream metrics and instead express all results as relative changes with respect to a task-specific reference. Throughout the paper, relative performance is computed as  $(x/\text{baseline} - 1)\%$ , where  $x$  is the score of the evaluated method.

### 3.1 Evaluation Protocol

We evaluate PRAGMA primarily via embedding probes and Low-Rank Adaptation (LoRA) (Hu et al., 2022) fine-tuning on downstream tasks.

#### 3.1.1 Embedding Probing

Embedding probing facilitates rapid iteration during experimentation before committing to LoRA fine-tuning, e.g., to gauge whether a new feature brings the expected gain, to select a checkpoint after a pre-training run for further evaluation, or to determine whether it is worth exploring a task as a downstream target at all. The embeddings are extracted from the History Encoder output ( $z_h$ ).



For our probing analysis, we evaluate the [USR] token, the final [EVT] token, and a combination of both, using a standard linear probe. Given a downstream task with predefined train, validation, and test partitions, we first forward each record through the frozen encoder to obtain fixed-size representations and then train a linear probe (logistic or linear regression) on the training partition. We observe that probe performance is robust to the choice of hyperparameters, so fitting a probe typically takes a couple of minutes. Since our architecture is inherently “pre-norm”, the embeddings were standard-scaled prior to probe fitting. We found that training the probe with the L-BFGS optimiser (Liu et al., 1989) yields the best results and converges quickly.

We note that while Gradient Boosted Decision Trees (GBDT) perform well on lower-dimensional embeddings (e.g., 192-d), the requirement for per-task hyper-parameter tuning and the increased time-to-fit make them less practical than linear probing for high-velocity model evaluation.

### 3.1.2 Downstream Adaptation with LoRA

To specialise the PRAGMA backbone for downstream tasks, we employ Low-Rank Adaptation (LoRA), which introduces a minimal parameter overhead of only 2–4 %. In this setup, the pre-trained weights are fine-tuned for task-specific objectives to bridge the gap between general representation learning and downstream requirements.

We apply LoRA to QKV projections and MLP layers within encoder layers, following a common practice (Hu et al., 2022; Dettmers et al., 2023), and default to rank = 8 with  $\alpha = 8$  across all experiments, but also sweep the rank across {4, 8, 16} on smaller datasets. We use the Adam optimiser (Kingma et al., 2015) for LoRA fine-tuning, and training typically uses 1/8 of the wall-clock time used during pre-training, converging in 12 hours to a few days depending on the dataset size.

### 3.1.3 Preparing Downstream Datasets

For each downstream task, we obtain a unique identifier, which typically consists of a profile id and an evaluation point. Next, we gather the event history and profile attributes directly preceding the evaluation point. We follow the pre-defined folds and splits for each downstream task. The downstream dataset collection process mirrors that of the pre-training dataset.

## 3.2 Downstream Tasks

**Credit Scoring.** The task is to assess credit risk for retail applications by predicting the probability of default within the first 12 months of use. The downstream dataset spans multiple years and is diverse across records. This task is cast as a binary classification problem with a minority class, and performance is measured with ROC-AUC and PR-AUC offline metrics.

**Communication Engagement.** The task is to predict whether a user who abandoned a credit application mid-process will open a re-engagement communication. This action serves as an upper-funnel proxy for resuming the application and eventually originating a loan. A distinguishing aspect of this task is the severely limited sample size, requiring the model to capture nuanced event-level signals from minimal data. This task is formulated as a binary classification problem, and the main offline metrics are ROC-AUC and PR-AUC.

**External Fraud.** This task is a representative fraud detection use case formulated as a binary classification problem. Performance is evaluated using precision and recall as the primary offline metrics.

**Product Recommendation.** The task is to predict which products a user is likely to adopt in the near future, conditioned on receiving a specific communication (e.g., email or push notification). A key challenge lies in modelling conversion propensity across multiple products simultaneously while accounting for the contextual influence of the communication. The task is formulated as a multilabel classification problem, where the model outputs independent probabilities of conversion for each product in the portfolio. Performance is evaluated using mean average precision (mAP) as the primary offline metric.

**Recurrent Transactions.** This task focuses on predicting whether a given transaction corresponds to a recurring subscription that will repeat in the following month. A key challenge lies in distinguishing true recurring patterns

| Task             | Metric    | Baseline (ref.) | PRAGMA   |
|------------------|-----------|-----------------|----------|
| Credit scoring   | PR-AUC    | –               | +130.2 % |
|                  | ROC-AUC   | –               | +12.4 %  |
| Comm. engagement | PR-AUC    | –               | +79.4 %  |
|                  | ROC-AUC   | –               | +20.4 %  |
| External fraud   | Precision | –               | +16.7 %  |
|                  | Recall    | –               | +64.7 %  |
| Product rec.     | mAP       | –               | +40.5 %  |
| Recurrent txns   | $F_1$     | –               | +5.8 %   |
| Lifetime value   | PR-AUC    | –               | +1.8 %   |
|                  | ROC-AUC   | –               | +2.6 %   |

Table 2: **PRAGMA significantly outperforms internal task-specific models while sharing most of the parameters across tasks.** The relative performance is computed as  $(\text{PRAGMA}/\text{baseline} - 1)$ . The large variant with LoRA fine-tuning is used as PRAGMA.

from irregular or one-off payments given limited historical signals. The problem is formulated as a binary classification task, and performance is evaluated using macro-averaged  $F_1$ -score to account for class imbalance and ensure balanced performance across classes.

**Lifetime Value (LTV).** The LTV task is to assess the probability of a user generating positive gross profit, and is formulated as a binary classification problem. A distinguishing aspect of the LTV dataset is that users have shorter event histories, e.g., a couple of weeks, while the prediction horizon is typically 6 months or more. The main offline metrics are ROC-AUC and PR-AUC.

### 3.3 Main Results

The results presented in Table 2 demonstrate that PRAGMA consistently outperforms existing task-specific baselines across nearly all evaluated domains, despite sharing most of its parameters across tasks. The most striking improvements are observed in precision-recall metrics for high-impact tasks: PR-AUC increased by 130.2 % in Credit Scoring and 79.4 % in Communication Engagement, suggesting that PRAGMA is exceptionally effective at identifying low-frequency, high-value signals where traditional models struggle. While ROC-AUC gains are more tempered, they remain substantial at +12.4 % and +20.4 % for the same tasks, respectively. Although performance is more comparable on tasks like Lifetime Value and Recurrent Transactions, the overall trend confirms that PRAGMA provides a superior universal representation that matches or exceeds the performance of isolated, task-specific models.

#### 3.3.1 Effect of Model Scale

The results in Table 3 illustrate the performance impact of scaling the PRAGMA architecture from the Small (S, 10 M) variant to the Medium (M, 100 M) and Large (L, 1 B) variants. We observe that scaling gains are highly task-dependent, with the most significant improvements concentrated in Credit Scoring, where the Large model achieves a +35.2 % boost in PR-AUC and a +5.8 % gain in ROC-AUC over the Small reference.

Notably, the scaling behaviour for Communication Engagement is non-monotonic; the Medium variant exhibits a slight ROC-AUC regression (−1.8 %), while the Large variant recovers to +0.7 %. For more stable metrics like Recurrent Transactions and LTV, performance gains are more modest, typically remaining under +3.5 %. These results suggest that while increasing parameter count generally enhances predictive power, the Small model already provides a highly competitive representation for transactional and lifetime value predictions, offering a potential efficiency sweet spot for those specific production use cases.

| Task             | Metric    | PRAGMA   |         |         |
|------------------|-----------|----------|---------|---------|
|                  |           | S (ref.) | M       | L       |
| External fraud   | Precision | –        | +12.0 % | +16.4 % |
|                  | Recall    | –        | +24.8 % | +23.5 % |
| Product rec.     | mAP       | –        | +18.9 % | +27.0 % |
| Credit scoring   | PR-AUC    | –        | +16.3 % | +35.2 % |
|                  | ROC-AUC   | –        | +3.6 %  | +5.8 %  |
| Lifetime value   | PR-AUC    | –        | +1.5 %  | +3.0 %  |
|                  | ROC-AUC   | –        | +1.7 %  | +3.4 %  |
| Comm. engagement | PR-AUC    | –        | +0.1 %  | +1.6 %  |
|                  | ROC-AUC   | –        | –1.8 %  | +0.7 %  |
| Recurrent txns   | $F_1$     | –        | +0.6 %  | +0.4 %  |

Table 3: **Model performance scales with parameter count.** The performance is relative to PRAGMA-S fine-tuned with LoRA and computed as  $(\text{model}/\text{PRAGMA-S} - 1)$ .

| Task             | Metric  | PRAGMA-M       |         |
|------------------|---------|----------------|---------|
|                  |         | Scratch (ref.) | LoRA    |
| Comm. engagement | PR-AUC  | –              | +18.6 % |
|                  | ROC-AUC | –              | +5.0 %  |
| Credit scoring   | PR-AUC  | –              | +13.0 % |
|                  | ROC-AUC | –              | +1.6 %  |
| Product rec.     | mAP     | –              | +10.3 % |
| Recurrent txns   | $F_1$   | –              | +0.6 %  |
| Lifetime value   | PR-AUC  | –              | +0.4 %  |
|                  | ROC-AUC | –              | +0.3 %  |

Table 4: **Performance comparison of LoRA fine-tuning against task-specific models trained from scratch.** Relative performance is computed as  $(\text{LoRA}/\text{Scratch} - 1)$ . LoRA consistently matches or exceeds the performance of full-parameter training from scratch.

### 3.3.2 Effect of Pre-training

The results in Table 4 validate our approach, demonstrating that LoRA fine-tuning consistently matches or exceeds the performance of full-parameter training from scratch across all evaluated tasks. The largest gains are observed in Communication Engagement, where LoRA achieves +18.6 % in PR-AUC and +5.0 % in ROC-AUC, suggesting that the pre-trained PRAGMA backbone captures rich diverse event patterns that are difficult to learn when training a model from scratch on a single downstream task. Credit Scoring follows a similar pattern, with LoRA yielding a +13.0 % improvement in PR-AUC and a +1.6 % lift in ROC-AUC. Product Recommendation also benefits substantially, with a +10.3 % gain in mAP. For Recurrent Transactions and Lifetime Value, the improvements are more modest (+0.6 %  $F_1$ , and +0.4 % / +0.3 % PR-AUC / ROC-AUC respectively), indicating that the scratch-trained baselines already capture most of the task-relevant structure for these objectives, and LoRA fine-tuning maintains parity without regression. These findings are particularly significant for production environments, as they confirm that PRAGMA can consolidate multiple independent, high-maintenance models into a single shared system without sacrificing predictive accuracy, while maintaining a significantly smaller trainable parameter footprint.

| Task             | Metric    | PRAGMA-S |         | PRAGMA-M |         | PRAGMA-L |         |
|------------------|-----------|----------|---------|----------|---------|----------|---------|
|                  |           | Emb.     | LoRA    | Emb.     | LoRA    | Emb.     | LoRA    |
| Product rec.     | mAP       | –        | +57.2 % | –        | +68.4 % | –        | +68.1 % |
| External fraud   | Precision | –        | +30.8 % | –        | +29.8 % | –        | +23.8 % |
|                  | Recall    | –        | +27.4 % | –        | +24.5 % | –        | +13.3 % |
| Comm. engagement | PR-AUC    | –        | +72.9 % | –        | +49.7 % | –        | +54.1 % |
|                  | ROC-AUC   | –        | +16.9 % | –        | +11.2 % | –        | +13.5 % |
| Credit scoring   | PR-AUC    | –        | +18.0 % | –        | +20.4 % | –        | +10.3 % |
|                  | ROC-AUC   | –        | +0.2 %  | –        | +2.4 %  | –        | +1.5 %  |
| Recurrent txns   | $F_1$     | –        | +4.5 %  | –        | +3.2 %  | –        | +2.3 %  |
| Lifetime value   | PR-AUC    | –        | +3.6 %  | –        | +2.4 %  | –        | +2.9 %  |
|                  | ROC-AUC   | –        | +4.7 %  | –        | +3.4 %  | –        | +3.9 %  |

Table 5: **Relative improvement of LoRA-tuned models over embedding-only baselines across scales.** For each model size (S, M, L), the embedding-only variant is used as the reference (Emb). Performance gains are computed as  $(\text{LoRA}/\text{Emb} - 1)$ .

### 3.4 Additional Experiments and Ablations

#### 3.4.1 Effect of Low-Rank Adaptation

As shown in Table 5, across all evaluated tasks and model scales, the LoRA-tuned variants consistently outperform the embedding-only baselines, demonstrating the efficacy of parameter-efficient fine-tuning in capturing task-specific nuances that fixed embeddings may miss. The most substantial improvements are observed in Communication Engagement, where LoRA delivers a remarkable +72.9 % gain in PR-AUC for the Small model and maintains significant leads in the Medium and Large variants. In Credit Scoring, we see a peak relative improvement of +20.4 % in PR-AUC for the Medium model, suggesting that LoRA layers are particularly effective at this scale for complex classification. Gains in Recurrent Transactions and LTV are more modest, typically ranging from +2.3 % to +4.7 %.

#### 3.4.2 Effect of Profile State

Table 6 isolates the contribution of the Profile State Encoder (§2.3) by comparing the full PRAGMA-S model against a variant that removes the profile-state branch entirely, relying solely on event-level representations. The impact is strongly task-dependent. Credit Scoring benefits substantially, with a +31.8 % relative gain in PR-AUC and +4.9 % in ROC-AUC. The outsized PR-AUC improvement indicates that profile state is particularly valuable for identifying the minority default class, where static signals such as account tenure and onboarding characteristics provide discriminative context that event sequences alone cannot fully capture. In contrast, Lifetime Value shows more moderate gains of +2.2 % in PR-AUC and +2.0 % in ROC-AUC, suggesting that gross-profit likelihood is largely inferable from transactional patterns over the prediction horizon. Communication Engagement exhibits a slight PR-AUC regression (−3.0 %) alongside a marginal ROC-AUC gain (+1.3 %), indicating that re-engagement propensity is driven almost entirely by pre-drop-off event patterns rather than static user characteristics. These results validate the two-branch design of PRAGMA: the dedicated Profile State Encoder adds significant value for tasks where static profile state is informative, while the architecture degrades gracefully when those signals are less relevant.

#### 3.4.3 Communication Engagement (Uplift)

This task moves beyond conversion prediction to optimal treatment selection: the goal is to identify which messaging strategy best re-engages users with abandoned credit applications. The dataset is smaller in scale than our other downstream benchmarks, yet large-scale pre-training proves decisive, significantly outperforming a baseline trained on the limited in-domain data alone. As an uplift task, it also offers a distinct evaluation angle — PRAGMA is used as a frozen feature extractor feeding a meta-learner rather than being fine-tuned, isolating representational quality in the absence of task-specific adaptation.

| Task             | Metric    | PRAGMA-S          |         |
|------------------|-----------|-------------------|---------|
|                  |           | Event-only (ref.) | Full    |
| External fraud   | Precision | –                 | +46.8 % |
|                  | Recall    | –                 | +85.6 % |
| Credit scoring   | PR-AUC    | –                 | +31.8 % |
|                  | ROC-AUC   | –                 | +4.9 %  |
| Product rec.     | mAP       | –                 | +3.5 %  |
| Lifetime value   | PR-AUC    | –                 | +2.2 %  |
|                  | ROC-AUC   | –                 | +2.0 %  |
| Recurrent txns   | $F_1$     | –                 | +2.4 %  |
| Comm. engagement | PR-AUC    | –                 | –3.0 %  |
|                  | ROC-AUC   | –                 | +1.3 %  |

Table 6: **Profile state contributes substantially to tasks where static user characteristics are discriminative.** The relative performance is computed as  $(\text{Full}/\text{Event-only} - 1)$ .

| Task                      | Metric | Baseline (ref.) | PRAGMA   |
|---------------------------|--------|-----------------|----------|
| Comm. engagement (uplift) | AUUC   | –               | +163.7 % |
|                           | SNIPS  | –               | +10.8 %  |

Table 7: **Performance comparison of PRAGMA-L against the internal uplift baseline using the same meta-learner framework.** The relative performance is computed as  $(\text{PRAGMA-L}/\text{Baseline} - 1)$ .

Concretely, we adopt a meta-learner framework (Künzel et al., 2019) to estimate heterogeneous treatment effects, requiring the model to capture complex interactions between pre-drop-off event signals, profile state, and treatment assignment. Both PRAGMA and the baseline use the same meta-learner, differing only in the underlying representation.

Table 7 summarises results using Area Under the Uplift Curve (AUUC) and SNIPS (Swaminathan et al., 2015). PRAGMA-L’s ability to capture latent event-level patterns translates to highly effective treatment allocation, achieving a relative AUUC increase of 163.7 % over the internal baseline.

### 3.4.4 Effect of a Pre-trained Text Encoder

In the standard PRAGMA architecture, text values are learned jointly with all other tabular features via an embedding lookup table (see §2.3.1). To prevent the model from underfitting sparse, noisy, or highly irregular financial text (e.g., truncated transaction descriptions), we investigate offloading text comprehension to a dedicated, pre-trained text embedding model, e.g., Nemotron-1B-v2 (de Souza P. Moreira et al., 2024). This decoupled approach provides richer, out-of-the-box semantics and frees the primary Event Transformer (§2.3.3) to focus on cross-feature interactions. While we do not use this as the default formulation in our generalized core architecture, we report on it as an optional extension that offers valuable domain-specific insights.

**Implementation Details.** The addition of a pre-trained text encoder involves multiple structural changes to the PRAGMA architecture. First, for semantic types (keys) whose values are normally encoded using a custom-trained BPE tokeniser and a trainable embedding lookup table, we instead use the frozen pre-trained model to map the complete text string to a single vector, which is then adapted via a one-layer trainable projection (see Figure 5). Second, instead of reconstructing exact token labels for these text fields during MLM optimisation (see §2.3.5), we train PRAGMA to reconstruct the continuous text embedding produced by the pre-trained text encoder with Mean Squared Error (MSE).

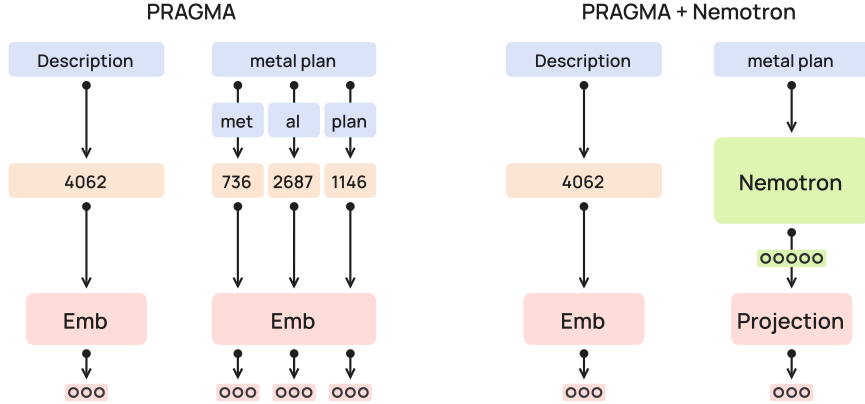


Figure 5: **Text embedding with PRAGMA (left) compared to a version with pre-trained Nemotron-1B-v2 text embedding (right).** Instead of our custom trained BPE tokeniser and a trainable embedding lookup table, a pre-trained “frozen” Nemotron maps an entire text value to a single text embedding vector which is projected into the Transformer’s base dimension with a trainable projection.

|                |           | PRAGMA-M |           |
|----------------|-----------|----------|-----------|
| Task           | Metric    | ref.     | +Nemotron |
| Credit scoring | PR-AUC    | –        | +16.1 %   |
|                | ROC-AUC   | –        | +2.8 %    |
| Recurrent txns | $F_1$     | –        | +0.1 %    |
| Lifetime value | PR-AUC    | –        | +0.8 %    |
|                | ROC-AUC   | –        | +0.6 %    |
| External fraud | Precision | –        | +3.8 %    |
|                | Recall    | –        | −0.7 %    |
| Product rec.   | mAP       | –        | −6.4 %    |

Table 8: **Impact of pre-trained text embeddings on downstream tasks is concentrated in text-heavy domains.** The performance is estimated relative to a LoRA-tuned PRAGMA-M.

**Results & Discussion.** The results are shown in Table 8. Downstream effects track how much label-relevant signal sits in free text versus categorical and behavioural structure. Credit Scoring shows the clearest upside, with +16.1 % relative PR-AUC and +2.8 % ROC-AUC under Nemotron. Product Recommendation instead loses ground: mAP drops by 6.4 % relative, plausibly because sparse text adds little beyond what the structural channels already encode. External Fraud moves modestly and in opposite directions on precision (+3.8 %) versus recall (−0.7 %), while LTV and Recurrent Transactions stay near flat on the reported metrics. Because this variant also increases PRAGMA-M training latency by about 18 %, we keep it as an opt-in module for text-heavy tasks rather than baking it into the default architecture.

### 3.4.5 Limitations in Highly Relational Tasks: Anti-Money Laundering

We formulate Anti-Money Laundering (AML) as a binary classification task. As shown in Table 9, this is a setting where PRAGMA significantly underperforms the production baseline.

We attribute this performance gap to two primary factors. First, the downstream AML dataset is sufficiently large for the baseline model to learn robust task-specific representations without requiring foundation-level pre-training. Second, and more critically, AML detection is inherently relational: the baseline leverages cross-record features that capture network-level signals. Because PRAGMA processes event histories in isolation, the resulting embeddings do not inherently capture the cross-record dependency structures crucial for this task.



| Task                  | Metric    | Baseline (ref.) | PRAGMA  |
|-----------------------|-----------|-----------------|---------|
| Anti-money laundering | $F_{0.5}$ | –               | –47.1 % |

Table 9: **Performance comparison of PRAGMA against baseline for Anti-Money Laundering.** The relative performance is computed as  $(\text{PRAGMA}/\text{Baseline} - 1)$  using linear probe on PRAGMA-L embeddings.

Performance is evaluated primarily using  $F_{0.5}$ , as it emphasises precision while still accounting for recall. PRAGMA suffers a 47.1 % drop in  $F_{0.5}$  compared to the network-aware baseline, demonstrating that isolated record-level representations may be insufficient for this highly relational domain. Addressing this limitation remains a key direction for future work.

## 4 Related Work

### 4.1 Transformer

The landscape of sequence modelling was fundamentally reshaped by the introduction of the Transformer architecture (Vaswani et al., 2017), which dispensed with recurrent layers in favour of a parallelisable self-attention mechanism. Following this, the field branched out into encoder-only models like BERT (Devlin et al., 2019), optimised for discriminative tasks, and decoder-only architectures like GPT-3 (Brown et al., 2020), which catalysed the current generative AI era through massive scaling and emergent in-context learning. Subsequent research has extended the architecture’s reach via the Vision Transformer (ViT) (Dosovitskiy et al., 2021) for visual perception and the T5 framework (Raffel et al., 2020) for unified text-to-text processing. Recent advancements have prioritised computational efficiency and multimodality, notably through hardware-aware optimisations like FlashAttention (Dao et al., 2022) and the adoption of Mixture-of-Experts (MoE) (Fedus et al., 2022) in models like Mixtral 8×7B (Jiang et al., 2024). In the current paradigm, models such as Gemini 1.5 (Gemini Team, 2024) and GPT-4o (Hurst et al., 2024) have moved beyond compositional architectures to native multimodality, enabling seamless reasoning across diverse data streams.

In this landscape, PRAGMA should be understood as an encoder foundation model for heterogeneous tabular event streams. Although motivated by financial transactions, it extends naturally to any domain where entities accumulate irregular, multi-field records over time. It inherits the scalability and bidirectional contextualisation of encoder-only Transformers, adapting them to heterogeneous fields, explicit time signals, and reusable record-level representations.

### 4.2 Masked Modelling

Parallel to the scaling of generative decoders, masked modelling established a dominant paradigm for self-supervised representation learning. This was pioneered by BERT (Devlin et al., 2019), which utilised a Masked Language Modelling (MLM) objective to capture bidirectional context, a technique further refined by RoBERTa (Liu et al., 2019) through dynamic masking and optimised training recipes. The success of MLM was later translated to the vision domain via Masked Image Modelling (MIM), with BEiT (Bao et al., 2021) and Masked Autoencoders (MAE) (He et al., 2022) demonstrating that reconstructing obscured image patches forces the model to learn holistic structural representations. Recent trends have moved towards cross-modal unification, as seen in Data2Vec (Baevski et al., 2022), and a shift from raw signal reconstruction to latent feature prediction, exemplified by the Joint-Embedding Predictive Architecture (I-JEPA) (Assran et al., 2023).

PRAGMA is directly inspired by this line of work, but extends masked modelling from text and images to heterogeneous financial records. Our objective masks individual tokens, whole events, and semantic types, encouraging the reconstruction of partially observed events and the learning of transferable representations from full transaction histories.

### 4.3 Transformers for Tabular Data

While Gradient Boosted Decision Trees (GBDTs) have historically dominated structured data, the Transformer has spurred a new class of “Tabular Deep Learning” architectures. Early entries like TabTransformer (Huang et al.,

2020) and FT-Transformer (Gorishniy et al., 2021) focused on modelling inter-feature dependencies through self-attention, demonstrating performance parity with GBDTs on high-dimensional datasets. This was improved by SAINT (Somepalli et al., 2021), which introduced a dual-attention mechanism for both feature and row interactions, and Tromp (Chen et al., 2023), which proposed prompt-tuning to disentangle intrinsic table properties from sample variations. A paradigm shift occurred with TabPFN (Hollmann et al., 2023), a foundation model pre-trained on synthetic data to approximate Bayesian inference. Leveraging in-context learning, TabPFN generates predictions via a single forward pass, eliminating the need for iterative training. While the original model was restricted to 1,000 samples, TabPFN-v2 and TabPFN-v2.5 (Hollmann et al., 2025; Grinsztajn et al., 2025) scaled the architecture to handle 100,000 samples and real-world complexities, providing native support for categorical features, missing values, and outliers. Most recently, Mitra (Zhang et al., 2025) has adopted the dual-attention mechanism of SAINT but follows the foundation model paradigm of TabPFN by being pre-trained exclusively on a massive mixture of synthetic priors.

PRAGMA is related in spirit to tabular Transformers because it preserves field identity and models cross-field interactions with attention, but unlike TabTransformer, FT-Transformer, and SAINT, it does not operate on a fixed-schema single row. Compared with TabPFN-style tabular foundation models trained on synthetic supervised tasks, PRAGMA is pre-trained with self-supervision on real financial ledgers and models variable-length user histories of heterogeneous events with a hierarchical encoder.

#### 4.4 Modelling for Recommender Systems

Sequential recommendation models share structural similarities with transaction modelling, as both process ordered event sequences with rich side information. Transformer-based recommenders treat user interaction histories as token sequences: SASRec (Kang et al., 2018) replaced recurrence with self-attention to capture long-range dependencies, and BERT4Rec (Sun et al., 2019) demonstrated that bidirectional context via masked item prediction yields more robust representations. The field later converged with the LLM paradigm: P5 (Geng et al., 2022) cast diverse recommendation tasks into a unified text-to-text framework built on T5, while TALLRec (Bao et al., 2023) introduced instruction tuning to align general-purpose LLMs with recommendation logic.

More recent industrial work has shifted from modelling only positive interactions to encoding richer event streams. Generative Recommenders (Zhai et al., 2024) interleave item and action tokens in a causal sequence, scaling to trillions of parameters with power-law quality gains. ARGUS (Khrylchenko et al., 2025) decomposes autoregressive learning into feedback and next-item prediction, scaling recommender Transformers to one billion parameters. The TransAct line of work (Xia et al., 2023; 2025) embeds each user action as a composite of content, action type, and context for CTR prediction, and extends to lifelong action sequences.

PRAGMA is close to this literature in its use of ordered event histories and self-supervised pre-training. Unlike recommendation models that often reduce each interaction to an item token, PRAGMA models richer financial events with typed fields, amounts, free text, and temporal coordinates, and is adapted to a broader set of banking tasks beyond ranking.

#### 4.5 Foundation Models for Finance

The paradigm of financial foundation models has rapidly matured from specialised text encoders to comprehensive reasoning engines that integrate diverse data modalities. This evolution began with FinBERT (Yang et al., 2020), which adapted the encoder-only architecture to financial corpora, establishing a rigorous baseline for discriminative tasks like sentiment analysis and ESG classification. The field shifted toward massive generative scale with BloombergGPT (Wu et al., 2023), which demonstrated that interleaving proprietary financial datasets with general web corpora yields superior performance on domain-specific benchmarks. To address the accessibility barriers of such massive models, FinGPT (Yang et al., 2023) introduced a data-centric, lightweight adaptation framework, democratising access to financial LLMs via efficient LoRA fine-tuning (Hu et al., 2022) of open-source models. Most recently, research has transcended textual boundaries to address the structured nature of market data; models like Time-LLM (Jin et al., 2024) and Chronos (Ansari et al., 2024) treat numerical time series as token sequences, enabling Transformers to perform zero-shot forecasting.

Extending this structural shift to consumer finance, recent foundation models are now being trained directly on massive-scale user transaction ledgers. For instance, nuFormer (Braithwaite et al., 2025) demonstrates that jointly

fusing tokenised transaction sequences with traditional tabular features can effectively replace manual feature engineering for real-world risk prediction. Concurrently, TransactionGPT (Dou et al., 2025) introduces a specialised 3D-Transformer architecture to explicitly model the multimodal, temporal, and tabular dimensions of billion-scale payment trajectories, achieving state-of-the-art performance in downstream anomaly detection and trajectory generation.

PRAGMA differs from text-centric financial foundation models such as FinBERT, BloombergGPT, and FinGPT, which primarily operate on financial language, and from Time-LLM or Chronos, which tokenise numerical time series for forecasting. It is closer to transaction-ledger models such as nuFormer and TransactionGPT, but aims for a reusable encoder backbone over multi-source banking events with explicit profile state and lightweight adaptation across diverse discriminative tasks.

## 5 Conclusion

We presented PRAGMA, a family of encoder-style foundation models for multi-source banking user histories. PRAGMA combines a key–value–time tokenisation scheme with two encoder branches for profile state and events whose outputs are fused by a history encoder, and is pre-trained with masked modelling on large-scale, heterogeneous financial records. Across diverse downstream tasks—credit scoring, fraud detection, communication engagement, product recommendation, recurrent transaction detection, lifetime value prediction, and more—a single pre-trained backbone achieves superior performance directly from raw banking event sequences, providing a general-purpose representation layer for financial applications.

Our experiments reveal several practical insights. LoRA fine-tuning consistently matches or exceeds full training from scratch while updating only a small fraction of parameters, confirming that the pre-trained representations transfer effectively across tasks. Scaling from 10 M to 1 B parameters yields large gains on harder tasks such as credit scoring, while smaller models already provide competitive representations for tasks such as lifetime value prediction, offering a practical efficiency trade-off. The dedicated profile state encoder proves particularly valuable for tasks where static contextual attributes are informative, such as credit scoring and fraud detection, while the architecture degrades gracefully when those signals are less relevant. We also find that integrating a pre-trained text encoder improves performance in text-dense domains but adds training overhead that is not justified for text-sparse tasks. Finally, the AML case study highlights a clear limitation: tasks that depend on cross-record relational structure remain out of reach for a model that processes event histories in isolation.

These results suggest that multi-source banking event sequences admit transferable representations in much the same way as text and vision, despite their heterogeneous structure, irregular timing, and operational constraints. Extending the model to capture cross-record interactions for relational tasks such as anti-money laundering is a promising direction for future work.

## Acknowledgments

We thank Dmitry Mittov, Ian Iakobsen, Aleksandr Pushin, Muhammad Anas, Viacheslav Karpov, Nathalie Skrzypek, Leyla Sultanova, Francisco Sanz Estevez, Nikita Kravchuk, Tadas Krisciunas, Amey Baokar, Hanna Danilovich, Jyoti Prakash Bal, Vitalii Radchenko, Kade Main, Nic Hatia, and other Revoluters for their contributions to this work.

## References

- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.
- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Conference on Computer Vision and Pattern Recognition*, 2023.
- Alexei Baevski, Wei-Ning Hsu, Qiantong Xu, Arun Babu, Jiatao Gu, and Michael Auli. Data2vec: A general framework for self-supervised learning in speech, vision and language. In *International Conference on Machine Learning*, 2022.

- Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. BEiT: BERT pre-training of image transformers. *arXiv preprint arXiv:2106.08254*, 2021.
- Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *ACM Conference on Recommender Systems*, 2023.
- Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue et al. On the opportunities and risks of foundation models. *ArXiv*, 2021.
- DT Braithwaite, Misael Cavalcanti, R Austin McEver, Hiroto Udagawa, Daniel Silva, Rohan Ramanath, Felipe Menezes, Arissa Yoshida, Evan Wingert, Matheus Ramos, et al. Your spending needs attention: Modeling financial habits with transformers. *arXiv preprint arXiv:2507.23267*, 2025.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 2020.
- Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *International Conference on Computer Vision*, 2021.
- Kuan-Yu Chen, Ping-Han Chiang, Hsin-Rung Chou, Ting-Wei Chen, and Darby Tien-Hao Chang. Trompt: Towards a better deep neural network for tabular data. In *International Conference on Machine Learning*, 2023.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 2022.
- Gabriel de Souza P. Moreira, Radek Osmulski, Mengyao Xu, Ronay Ak, Benedikt Schifferer, and Even Oldridge. Nv-retriever: Improving text embedding models with effective hard-negative mining. *arXiv preprint arXiv:2407.15831*, 2024. doi: 10.48550/arXiv.2407.15831.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 2023.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *North American Association for Computational Linguistics - Human Language Technologies*, 2019.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- Yingtong Dou, Zhimeng Jiang, Tianyi Zhang, Mingzhi Hu, Zhichao Xu, Shubham Jain, Uday Singh Saini, Xiran Fan, Jiarui Sun, Menghai Pan, et al. Transactiongpt. *arXiv preprint arXiv:2511.08939*, 2025.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 2022.
- Gemini Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *ACM Conference on Recommender Systems*, 2022.
- Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems*, 2021.

- Yury Gorishniy, Ivan Rubachev, and Artem Babenko. On embeddings for numerical features in tabular deep learning. *Advances in Neural Information Processing Systems*, 2022.
- Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Benjamin Jäger, Dominik Safaric, Simone Alessi, Adrian Hayler, et al. TabPFN-2.5: Advancing the state of the art in tabular foundation models. *arXiv preprint arXiv:2511.08667*, 2025.
- Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Computer Vision and Pattern Recognition*, 2022.
- Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations*, 2023.
- Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 2025.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. TabTransformer: Tabular data modeling using contextual embeddings. *arXiv preprint arXiv:2012.06678*, 2020.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuanfang Li, Shirui Pan, and Qingsong Wen. Time-LLM: Time series forecasting by reprogramming large language models. In *International Conference on Learning Representations*, 2024.
- Keller Jordan. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *International Conference on Data Mining*, 2018.
- Kirill Khrylchenko, Artem Matveev, Sergei Makeev, and Vladimir Baikalov. Scaling recommender transformers to one billion parameters. *arXiv preprint arXiv:2507.15994*, 2025.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Computer Vision and Pattern Recognition*, 2023.
- Sören R Künzel, Jasjeet S Sekhon, Peter J Bickel, and Bin Yu. Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the national academy of sciences*, 2019.
- Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 1989.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025.

- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 2020.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Annual Meeting of the Association for Computational Linguistics*, 2016.
- Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C Bayan Bruss, and Tom Goldstein. SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training. *arXiv preprint arXiv:2106.01342*, 2021.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 2014.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.
- Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *International Conference on Information and Knowledge Management*, 2019.
- Adith Swaminathan and Thorsten Joachims. The self-normalized estimator for counterfactual learning. In *NeurIPS*, 2015.
- Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Computer Vision and Pattern Recognition*, 2016.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Ghaffari, Binyam Gebre, Abraham Ittycheriah, and Gideon Mann. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.
- Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. TransAct: Transformer-based realtime user action model for recommendation at Pinterest. In *ACM SIGKDD*, 2023.
- Xue Xia, Saurabh Vishwas Joshi, Kousik Rajesh, Kangnan Li, Yangyi Lu, Nikil Pancha, Dhruvil Deven Badani, Jiajing Xu, and Pong Eksombatchai. TransAct V2: Lifelong user action sequence modeling on Pinterest recommendation. *arXiv preprint arXiv:2506.02267*, 2025.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, 2020.
- Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. FinGPT: Open-source financial large language models. In *International Joint Conference on Artificial Intelligence (IJCAI) Symposium on Financial Large Language Models*, 2023.
- Yi Yang, Mark Christopher Siy Uy, and Allen Huang. FinBERT: A pretrained language model for financial communications. *arXiv preprint arXiv:2006.08097*, 2020.



Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152*, 2024.

Xiyuan Zhang, Danielle C. Maddix, Junming Yin, Nick Erickson, Abdul Fatir Ansari, Boran Han, Shuai Zhang, Leman Akoglu, Christos Faloutsos, Michael W. Mahoney, Cuixiong Hu, Huzefa Rangwala, George Karypis, and Bernie Wang. Mitra: Mixed synthetic priors for enhancing tabular foundation models. *Advances in Neural Information Processing Systems*, 2025.